MULTI-ARMED BANDITS

Exploration, Exploitation & Upper Confidence Bound

A visual, from-scratch-to-pro tutorial in Reinforcement Learning

01	The problem: slot machines & unknown rewards
02	Exploration vs. exploitation, explained simply
03	Greedy and ε-greedy strategies (with simulations)
04	Upper Confidence Bound (UCB1) — full derivation & intuition
05	Regret: the metric that ties it all together
06	Advanced: Bayesian bandits, Thompson Sampling, contextual bandits
07	Bonus: Pro tips for K-Means Clustering

Designed for dark-mode reading • Includes simulations, diagrams & math

PART 1 — FOUNDATIONS

0. A 60-Second Primer on Reinforcement Learning

Reinforcement Learning (RL) is the branch of machine learning where an agent learns to make decisions by interacting with an environment. At each step, the agent takes an action, the environment returns a reward, and the agent updates what it believes is the best behaviour. Unlike supervised learning, nobody tells the agent the "correct" action directly — it has to discover good actions through trial and error.

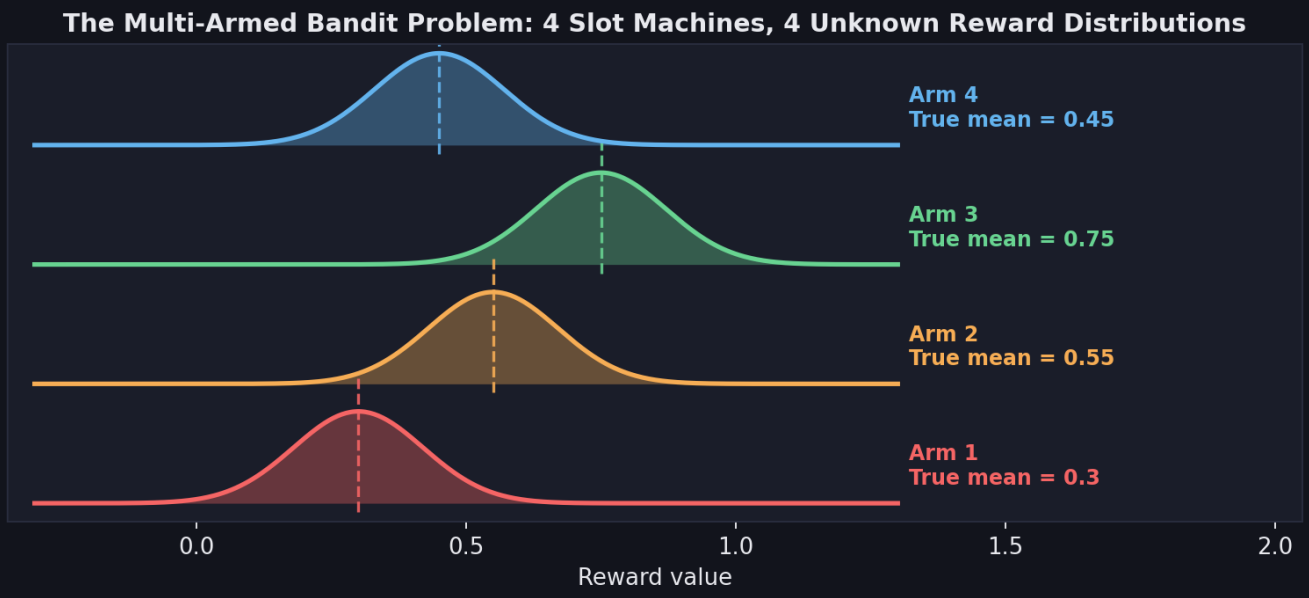
The Multi-Armed Bandit (MAB) problem is the simplest possible RL problem. It strips away almost everything — there is no notion of "state" changing over time, no sequences of moves that affect the future. There's just one decision, repeated many times: which action do I take right now to get the most reward? This simplicity is exactly what makes it the perfect starting point for understanding exploration and exploitation.

✓ **Why bandits matter in the real world**

- A/B testing websites: which button color gets more clicks?
- Ad placement: which advertisement earns the most revenue per impression?
- Clinical trials: which treatment helps patients most, while minimizing harm to trial participants?
- Recommendation systems: which movie/song to suggest next?

1. The Multi-Armed Bandit Problem

Imagine walking into a casino with a row of slot machines ("one-armed bandits"). Each machine pays out rewards according to its own hidden probability distribution — and you don't know what those distributions are. You have a fixed number of pulls (coins) to spend. Your job: pull the arms in a way that maximizes your total winnings.



Four arms, four hidden reward distributions. The agent must discover which pays best, purely from experience.

Formally, a k-armed bandit has k possible actions (arms). Each arm a has a true, unknown expected reward q*(a). At every time step t, the agent selects an arm A_t, and receives a reward R_t sampled from that arm's distribution.

Key definitions

```
q*(a) = E[ R_t | A_t = a ] (true value of action a — unknown to the agent)

Q_t(a) ≈ q*(a) (the agent's ESTIMATE of that value, at time t)

N_t(a) = number of times arm a has been chosen before time t
```

The agent never sees q*(a) directly. All it can do is keep a running estimate $Q_t(a)$, usually the simple average of the rewards observed from that arm so far:

Sample-average estimate

```
Q_t(a) = ( sum of rewards from a ) / ( N_t(a) )

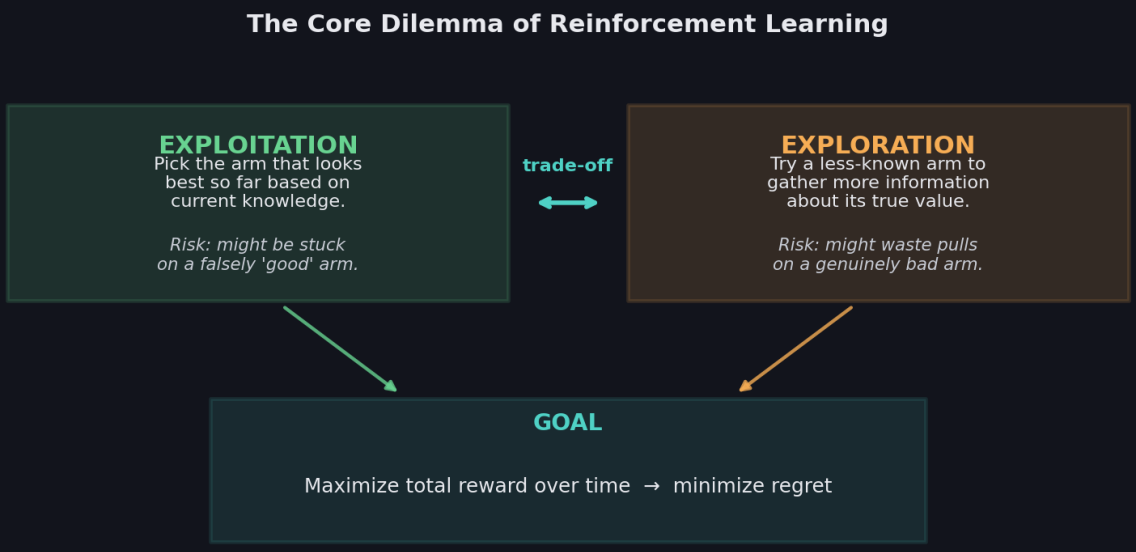
Incremental update (no need to store all past rewards):

Q_{n+1} = Q_n + (1 / n) * ( R_n − Q_n )
```

This incremental form is elegant: the new estimate is the old estimate, nudged toward the newest reward by an amount that shrinks as n grows. Early rewards move the estimate a lot; later rewards barely move it. This is the mathematical seed of almost every learning-rate idea in machine learning.

2. The Exploration vs. Exploitation Dilemma

Suppose after a few pulls, Arm 3 looks best in your running estimates. Do you keep pulling Arm 3 forever (exploit what you know), or do you occasionally try Arm 1, 2, or 4 in case your early estimate was misleading (explore to learn more)?



This is the central tension in reinforcement learning, not just in bandits. Pure exploitation risks getting permanently stuck on an arm that only looked good due to random luck early on. Pure exploration wastes pulls on arms you already know are mediocre. The right answer is somewhere in between, and how you strike that balance is what separates different bandit algorithms.

2.1 The naive approach: pure greedy

The simplest possible policy always exploits: pick the arm with the highest current estimate.

Greedy action selection

```
A_t = argmax_a Q_t(a)
```

The problem: if an inferior arm happens to give a lucky high reward on the very first pull, greedy will fixate on it and may never try the truly best arm again. This is why pure greedy usually performs poorly in the long run — it has zero built-in exploration.

2.2 A small fix: ε-greedy (epsilon-greedy)

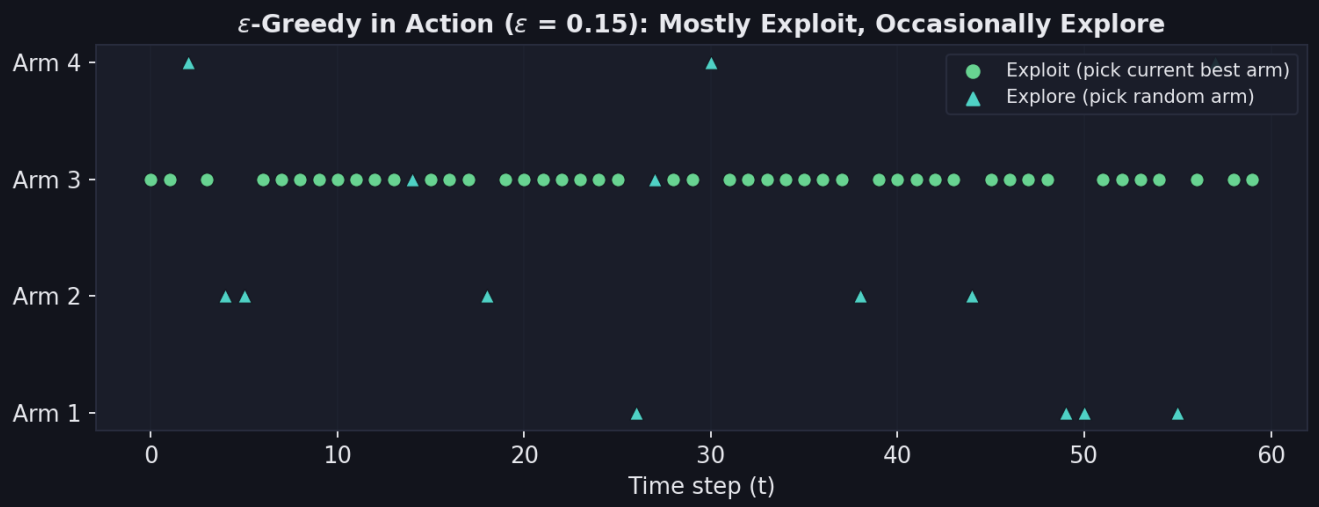
ε-greedy injects randomness: most of the time (probability $1-\varepsilon$) it exploits the best-known arm, but with small probability ε it picks a uniformly random arm instead — guaranteeing that,

given enough time, every arm keeps getting sampled.

ε-greedy policy

```
With probability (1 − ε): A_t = argmax_a Q_t(a) (exploit)

With probability ε: A_t = random arm (explore)
```



With ε = 0.15, the agent exploits roughly 85% of the time and explores the rest — shown here as scattered random picks (triangles).

✓ Choosing ε

- Larger ε (e.g. 0.3) explores more — good early on, or in highly non-stationary problems.
- Smaller ε (e.g. 0.01–0.1) exploits more — good once you're fairly confident in estimates.
- A common trick: start with a larger ε and decay it over time ("ε-decay").

⚠ The core weakness of ε-greedy

- It explores blindly — a random arm is picked with equal probability, even one that's clearly terrible.
- It never uses uncertainty. Trying an arm you've pulled 1,000 times is treated the same as one pulled 3 times.
- This is exactly the gap that UCB is designed to close.

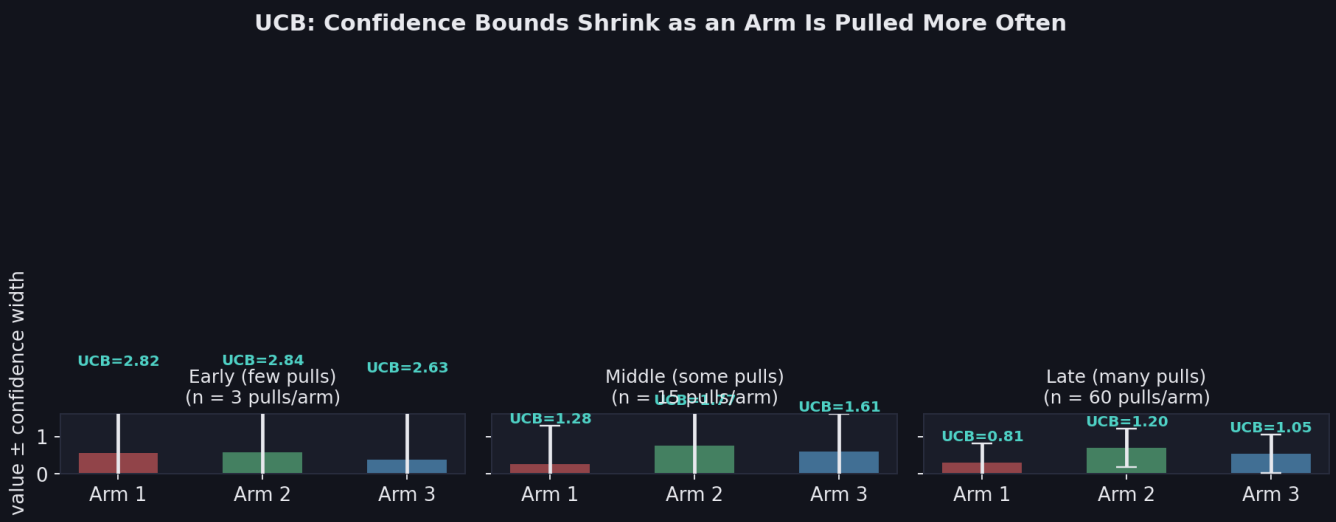
PART 2 — UPPER CONFIDENCE BOUND

3. Upper Confidence Bound (UCB): Explore Smartly, Not Randomly

UCB's key idea: instead of exploring randomly, explore optimistically. Give every arm the "benefit of the doubt" proportional to how uncertain you are about it. An arm you've tried only twice could secretly be great — so its optimistic estimate should be boosted more than an arm you've already tried 500 times. This principle is often summarized as "optimism in the face of uncertainty."

3.1 Confidence intervals shrink with more data

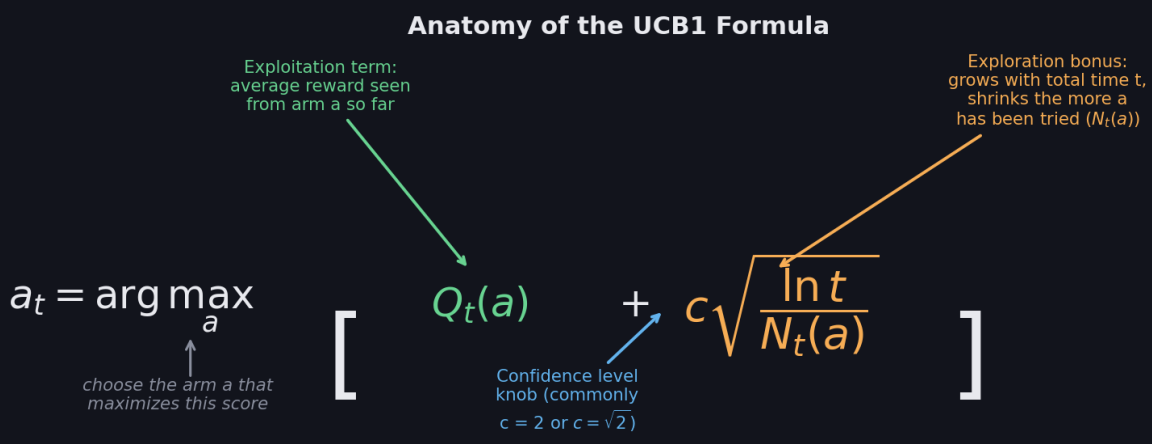
Every time you pull an arm, you learn a little more about its true value, and your uncertainty about it shrinks. Statistically, this uncertainty behaves like a confidence interval around your estimate — wide when you have few samples, narrow when you have many.



The bars are estimated values; the whiskers are the confidence width. As pulls increase (left to right), the whiskers shrink and estimates stabilize.

3.2 The UCB1 formula

UCB1 (the classic, most widely taught version) scores every arm as the sum of its current estimate plus an exploration bonus, then always picks the arm with the highest score.



UCB1 selection rule

```
A_t = argmax_a [ Q_t(a) + c * sqrt( ln(t) / N_t(a) ) ]

Q_t(a) : sample-average reward observed from arm a so far (exploitation)

N_t(a) : number of times arm a has been pulled (confidence)

t : total number of pulls made so far, across all arms

c : exploration constant (commonly c = 2, or c = sqrt(2) from Hoeffding's bound)
```

Notice what happens to any arm that hasn't been pulled in a while: t (the numerator, inside the log) keeps growing for every other pull made, while $N_t(a)$ (its own denominator) stays fixed. That means its exploration bonus keeps growing the longer it's ignored — eventually forcing UCB to go back and re-check it. This is a built-in, principled substitute for ε-greedy's random exploration.

✓ **Reading the formula intuitively**

- Q_t(a) alone would just be greedy — always the highest average.
- The square-root term is large when N_t(a) is small (few pulls) → strong incentive to try it.
- The square-root term shrinks toward 0 as N_t(a) grows → confidence replaces curiosity.
- ln(t) grows very slowly, so old, well-tested arms don't get boosted much even after a long time.

Where does sqrt(ln t / N) come from?

It comes from Hoeffding's inequality, a concentration bound from probability theory. Loosely, it tells us that with high probability, the true mean q*(a) lies within a "confidence radius" of the sample mean $Q_t(a)$:

```
P( q*(a) > Q_t(a) + sqrt( 2 ln(t) / N_t(a) ) ) ≤ 1 / t⁴ (Hoeffding bound, informal)
```

In practice, most implementations replace the theoretical constant with a tunable c to control how aggressively the algorithm explores.

Because this bound gets tighter as $N_t(a)$ grows, UCB is effectively always betting on the upper edge of where an arm's true value could plausibly be — hence the name "Upper Confidence Bound." If that optimistic upper edge turns out to be wrong, more pulls will correct it soon.

3.3 UCB1 step-by-step (pseudocode)

```
Initialize: for each arm a, N(a) = 0, Q(a) = 0

For t = 1, 2, 3, ... :
    If any arm has N(a) == 0:
        A_t = that unplayed arm      # try every arm once first
    Else:
        A_t = argmax_a [ Q(a) + c * sqrt( ln(t) / N(a) ) ]

    Pull arm A_t, observe reward R_t
    N(A_t) += 1
    Q(A_t)  = Q(A_t) + ( R_t - Q(A_t) ) / N(A_t)
```

3.4 Worked mini-example

Suppose after t = 10 pulls, three arms have these stats (c = 2):

Arm	N(a)	Q(a)	Exploration bonus = 2·√(ln 10 / N)	UCB score
A	5	0.40	2·√(2.303/5) = 1.36	1.76
B	2	0.55	2·√(2.303/2) = 2.15	2.70
C	3	0.60	2·√(2.303/3) = 1.75	2.35

Even though Arm B has the lowest raw pull count and a decent (not the highest) average, its huge exploration bonus — because it's been tried so few times — gives it the highest UCB score. UCB1 will pick Arm B next, purely to reduce uncertainty about it, even though Arm C currently "looks" almost as good on average.

4. Putting It All Together: A Simulation

Numbers convince better than words. Below, we simulate a 5-armed bandit for thousands of steps, averaged over hundreds of independent runs, comparing pure greedy, ε-greedy, and UCB1.



UCB1 (teal) converges fastest to picking the optimal arm and sustains the highest average reward. ε-greedy (orange) does well but always keeps some random exploration. Greedy (red) gets permanently stuck below optimal.

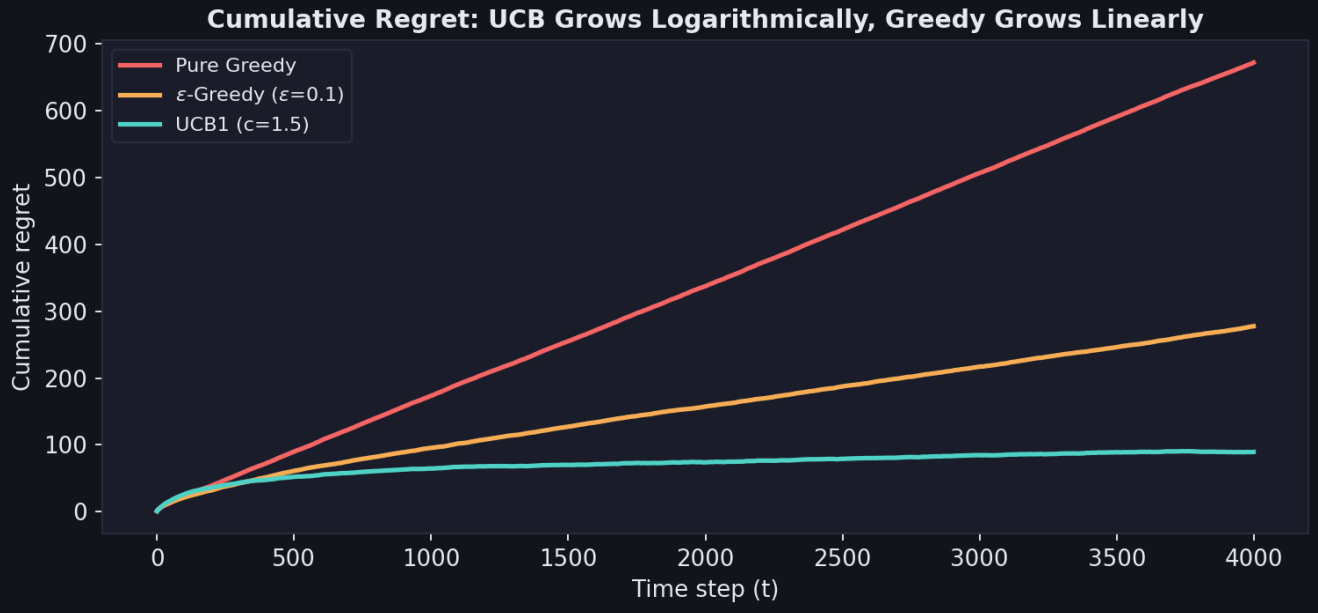
4.1 Regret: the single number that matters

Regret measures how much reward you lost, in total, compared to an oracle that always knew and picked the best arm from the very first step.

Regret definition

```
Regret(T) = T · q*(a*) − E[ sum_{t=1}^{T} R_t ]

where a* = argmax_a q*(a) is the (unknown, ideal) best arm
```



Pure greedy accumulates regret linearly (it never fully stops making mistakes). UCB1's regret grows only logarithmically — the theoretically optimal rate.

✓ Why logarithmic regret is a big deal

- Linear regret means average performance never improves — you keep losing the same amount, forever.
- Logarithmic regret means mistakes become extremely rare as t grows — the algorithm is 'learning'.
- UCB1 is provably within a constant factor of optimal regret for this problem class (Auer et al., 2002).

PART 3 — GOING FURTHER

5. Advanced Territory: Beyond UCB1

5.1 UCB variants

- UCB1-Tuned: replaces the fixed variance assumption with an empirical estimate of each arm's variance, usually tightening the bound in practice.
- UCB2: groups exploration into epochs to reduce switching costs between arms.
- KL-UCB: uses Kullback-Leibler divergence instead of Hoeffding's bound for a tighter, distribution-aware confidence radius — often the best-performing UCB variant for Bernoulli rewards.

5.2 Thompson Sampling (Bayesian bandits)

Instead of a confidence-bound formula, Thompson Sampling maintains a full probability distribution (a belief) over each arm's true value, updates it with Bayes' rule after every pull, and picks an arm by literally drawing a random sample from each arm's belief distribution and choosing the highest sample.

Thompson Sampling (Bernoulli rewards, Beta prior)

```
Prior belief for arm a: Beta( α_a , β_a ) (start with α=β=1, i.e. uniform)

After observing reward r ∈ {0,1}:

if r = 1: α_a ← α_a + 1

if r = 0: β_a ← β_a + 1

Action selection: sample θ_a ~ Beta(α_a, β_a) for every arm, pick argmax_a θ_a
```

In practice, Thompson Sampling often matches or beats UCB1 empirically, especially with delayed or noisy feedback, and it naturally supports non-stationary problems by decaying old evidence.

5.3 Contextual bandits

Real systems (ad servers, recommenders) rarely face the exact same decision every time — they see some context x_t (e.g., user features) before deciding. A contextual bandit chooses an arm as a function of the context: $A_t = \pi(x_t)$. Algorithms like LinUCB extend UCB by fitting a linear model of expected reward per arm as a function of context, then applying a UCB-style bonus on top of the model's own prediction uncertainty.

5.4 Non-stationary bandits

If reward distributions drift over time (e.g., user tastes evolve), a plain running average becomes stale. Two common fixes:

- Constant step-size updates: $Q \leftarrow Q + \alpha(R - Q)$ with fixed α, so recent rewards matter more than old ones.
- Sliding-window / discounted UCB: only count recent pulls (or discount old ones exponentially) inside $N_t(a)$ and $Q_t(a)$.

5.5 From bandits to full Reinforcement Learning

The multi-armed bandit is a stateless, one-step decision problem. Full RL (Markov Decision Processes) adds state transitions: the action you take now changes what situation you're in next, and rewards can be delayed. Exploration-exploitation is still central there — algorithms like UCB-based exploration bonuses and Thompson-Sampling-flavoured posterior sampling reappear inside deep RL agents (e.g., UCB exploration terms in Monte-Carlo Tree Search, used in AlphaGo/AlphaZero).

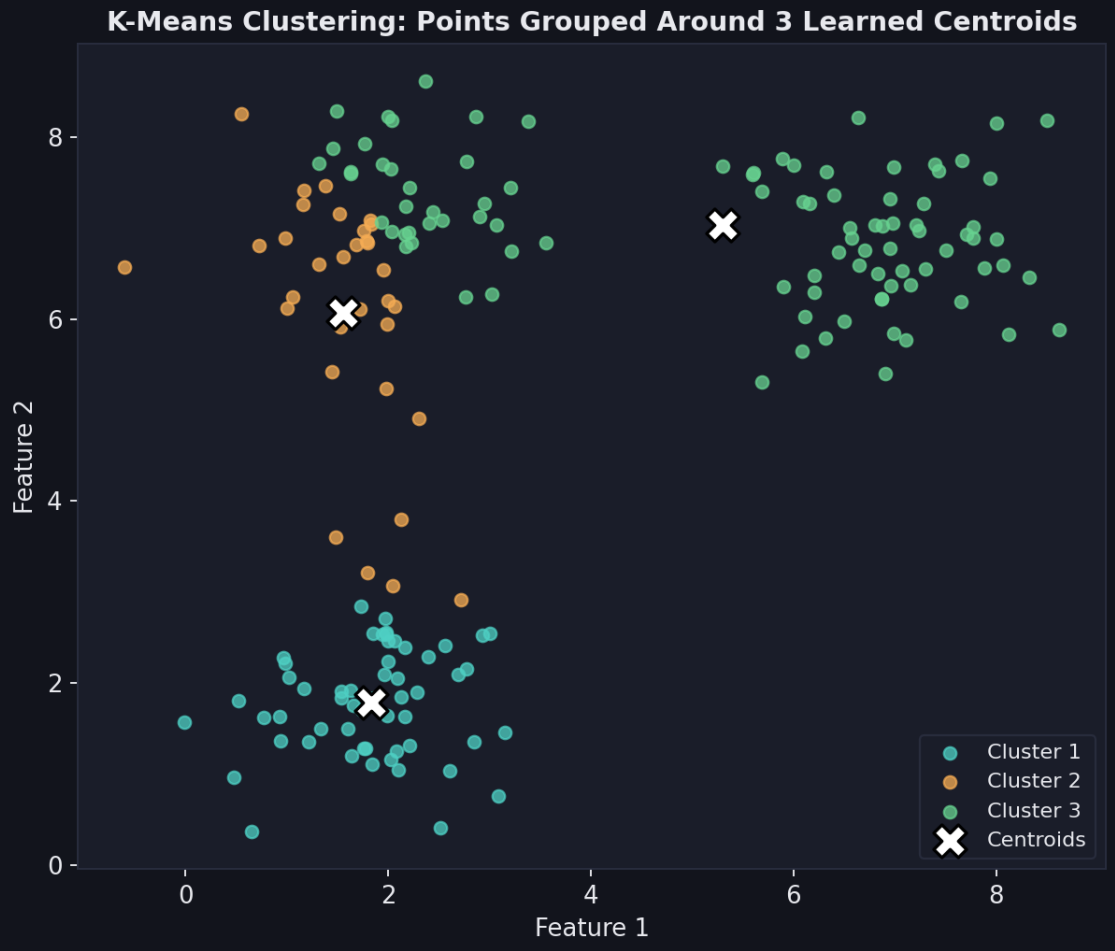
✓ **Mental model to keep forever**

- Bandits = 1-step decisions, no state. Full RL = many-step decisions, state matters.
- Every RL agent, no matter how advanced, still faces some version of 'explore vs exploit' at its core.
- UCB's 'optimism under uncertainty' principle shows up everywhere: bandits, tree search, even Bayesian optimization.

BONUS

6. Pro Tips for K-Means Clustering

K-Means is an unsupervised algorithm that groups data points into k clusters by iteratively assigning each point to its nearest centroid, then recomputing centroids as the mean of their assigned points, until assignments stop changing.



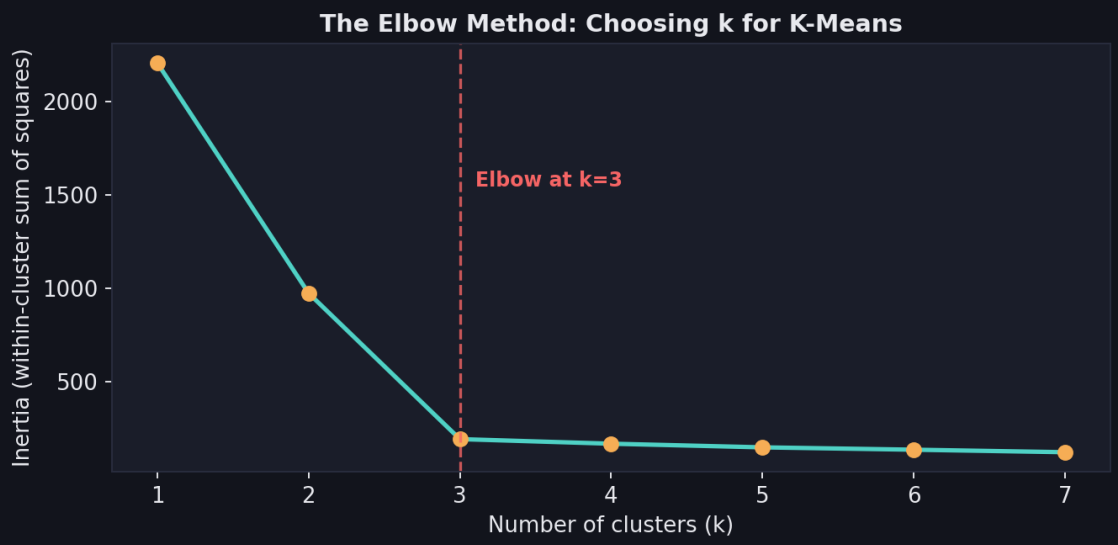
Three well-separated clusters (colors) with their learned centroids (white X markers).

K-Means objective (inertia)

```
minimize J = sum_{i=1}^{k} sum_{x in Cluster_i} || x − μ_i ||²

μ_i = centroid (mean) of cluster i, ||·|| = Euclidean distance
```

6.1 Choosing k



The Elbow Method: inertia keeps dropping as k grows, but the 'elbow' (here, k=3) marks diminishing returns.

- Elbow method: plot inertia vs. k, look for the bend where adding more clusters stops helping much.
- Silhouette score: measures how similar a point is to its own cluster vs. other clusters (range −1 to 1) — more robust than the elbow method alone.
- Domain knowledge: often the best guide (e.g., you know you want exactly 3 customer segments).

6.2 Practical pro tips

✓ Before you run K-Means

- Always scale/standardize features first (e.g., StandardScaler). K-Means uses Euclidean distance, so unscaled features with large ranges dominate the result.
- Remove or cap extreme outliers — a single far-away point can drag an entire centroid toward it.
- Check for categorical variables; K-Means needs numeric, continuous-like features (consider K-Modes/K-Prototypes for mixed data).

✓ While running it

- Use k-means++ initialization (the default in scikit-learn) instead of random centroids — it drastically reduces the chance of poor local minima.
- Run with multiple random initializations (n_init) and keep the lowest-inertia result — K-Means is sensitive to starting points.
- Set a random_state for reproducibility during experimentation.

✓ **Interpreting & validating results**

- Don't fully trust inertia alone to compare different k values across different feature sets — it always decreases as k grows.
- Visualize clusters in 2D via PCA or t-SNE if you have many features, to sanity-check that clusters look sensible.
- Profile each cluster's average feature values afterward — the real business value is often in explaining what makes each cluster unique.

⚠ **Common pitfalls**

- K-Means assumes roughly spherical, similarly-sized clusters — it struggles with elongated or very differently-sized/shaped groups.
- It's sensitive to outliers because it minimizes squared distance (outliers get squared, amplifying their pull).
- It's a local optimization method: different runs can converge to different results. Never trust a single run for important decisions.

6.3 When to reach for something else

- DBSCAN / HDBSCAN: for irregularly shaped clusters and automatic outlier/noise detection, no need to pre-specify k.
- Gaussian Mixture Models (GMM): when clusters can overlap or have different shapes/sizes (soft, probabilistic assignment instead of hard).
- Hierarchical clustering: when you want a dendrogram showing nested cluster structure at multiple resolutions.

End of tutorial. You went from the basic slot-machine analogy, through ε-greedy, the full UCB1 formula and its Hoeffding-bound origins, regret analysis, Thompson Sampling and contextual bandits, all the way to practical K-Means tips. Revisit the simulation plots whenever the math feels abstract — they are the same algorithms, just made visible.